

45 Days Summer Training Program

Assignment :- 05

1. Create a 1D NumPy array of 20 random integers between 1 and 100.

Find the minimum, maximum, mean, and standard deviation without using Python built-in functions — use only NumPy aggregation functions.

➤ `import numpy as np`

```
arr = np.random.randint(1, 101, 20)
```

```
minimum = np.min(arr)
```

```
maximum = np.max(arr)
```

```
mean = np.mean(arr)
```

```
std_deviation = np.std(arr)
```

```
print("Array:", arr)
```

```
print("Minimum value:", minimum)
```

```
print("Maximum value:", maximum)
```

```
print("Mean:", mean)
```

```
print("Standard Deviation:", std_deviation)
```

- output :-

Array: [12 85 34 67 91 23 56 78 45 10 99 38 72 15 64
28 81 49 53 60]

Minimum value: 10

Maximum value: 99

Mean: 53.0

Standard Deviation: 27.07

2. Create a 4×4 matrix using NumPy. Perform the following operations:

- Extract the diagonal elements
- Replace all even numbers with 0
- Find the index of the maximum value using `argmax()`

➤ `import numpy as np`

```
arr = np.array([[10, 25, 8, 17],  
                [32, 45, 56, 71],  
                [19, 84, 63, 28],  
                [90, 11, 37, 52]])
```

```
diagonal = np.diag(arr)
```

```
arr[arr % 2 == 0] = 0
```

```
max_index = np.argmax(arr)
```

```
print("Matrix:\n", arr)
```

```
print("Diagonal Elements:", diagonal)
```

```
print("Index of Maximum Value:", max_index)
```

- output :-

Matrix:

```
[[ 0 25  0 17]
 [ 0 45  0 71]
 [19  0 63  0]
 [ 0 11 37  0]]
```

Diagonal Elements: [10 45 63 52]

Index of Maximum Value: 7

3. Write a program to create two 3×3 NumPy arrays and perform element-wise addition, subtraction, and matrix multiplication. Display results with proper labels.

➤ `import numpy as np`

```
a = np.array([[1, 2, 3],
              [4, 5, 6],
              [7, 8, 9]])
```

```
b = np.array([[9, 8, 7],
              [6, 5, 4],
              [3, 2, 1]])
```

```
add = a + b
```

```
sub = a - b
mul = np.matmul(a, b)

print("First Array:\n", a)
print("\nSecond Array:\n", b)
print("\nElement-wise Addition:\n", add)
print("\nElement-wise Subtraction:\n", sub)
print("\nMatrix Multiplication:\n", mul)
```

- Output:-

First Array:

```
[[1 2 3]
 [4 5 6]
 [7 8 9]]
```

Second Array:

```
[[9 8 7]
 [6 5 4]
 [3 2 1]]
```

Element-wise Addition:

```
[[10 10 10]
 [10 10 10]
 [10 10 10]]
```

Element-wise Subtraction:

```
[[ -8 -6 -4]
 [ -2  0  2]]
```

```
[ 4  6  8]
```

Matrix Multiplication:

```
[[ 30  24  18]  
 [ 84  69  54]  
 [138 114  90]]
```

4. Using `np.linspace()` and `np.zeros()` and `np.ones()`, create three different arrays and stack them vertically using `np.vstack()`. Display the final stacked array.

```
➤ import numpy as np
```

```
a = np.linspace(1, 10, 5)
```

```
b = np.zeros(5)
```

```
c = np.ones(5)
```

```
stacked = np.vstack((a, b, c))
```

```
print(stacked)
```

- output :-

```
[[ 1.  3.25  5.5  7.75 10. ]
```

```
 [ 0.  0.  0.  0.  0. ]
```

```
 [ 1.  1.  1.  1.  1. ]]
```

Section B – Pandas

5. Create a DataFrame of at least 10 students with columns: Name, Age, Marks, City. Perform the following:

- Display the first 5 rows using head()
- Show basic statistics using describe()
- Filter students who scored more than 75 marks
- Sort by Marks in descending order

```
➤ import pandas as pd
```

```
data = {  
    "Name": ["Aman", "Riya", "Karan", "Sneha",  
            "Rahul",  
            "Priya", "Vikas", "Neha", "Arjun", "Pooja"],  
    "Age": [18, 19, 20, 18, 21, 19, 20, 18, 21, 19],  
    "Marks": [82, 67, 91, 74, 88, 79, 65, 95, 72, 84],  
    "City": ["Delhi", "Mumbai", "Jaipur", "Pune",  
            "Chennai",  
            "Kolkata", "Hyderabad", "Bangalore",  
            "Lucknow", "Ahmedabad"]  
}
```

```
df = pd.DataFrame(data)
```

```
print("First 5 Rows:")  
print(df.head())
```

```
print("\nBasic Statistics:")  
print(df.describe())
```

```
print("\nStudents Scoring More Than 75 Marks:")  
print(df[df["Marks"] > 75])
```

```
print("\nSorted by Marks (Descending):")  
print(df.sort_values(by="Marks",  
ascending=False))
```

- output :-

First 5 Rows:

	Name	Age	Marks	City
0	Aman	18	82	Delhi
1	Riya	19	67	Mumbai
2	Karan	20	91	Jaipur
3	Sneha	18	74	Pune
4	Rahul	21	88	Chennai

Basic Statistics:

	Age	Marks
count	10.000000	10.000000
mean	19.300000	79.700000
std	1.159502	10.477488
min	18.000000	65.000000
25%	18.250000	72.500000
50%	19.000000	80.500000
75%	20.000000	87.000000
max	21.000000	95.000000

Students Scoring More Than 75 Marks:

	Name	Age	Marks	City
0	Aman	18	82	Delhi
2	Karan	20	91	Jaipur
4	Rahul	21	88	Chennai
5	Priya	19	79	Kolkata
7	Neha	18	95	Bangalore
9	Pooja	19	84	Ahmedabad

Sorted by Marks (Descending):

	Name	Age	Marks	City
7	Neha	18	95	Bangalore
2	Karan	20	91	Jaipur
4	Rahul	21	88	Chennai
9	Pooja	19	84	Ahmedabad
0	Aman	18	82	Delhi
5	Priya	19	79	Kolkata
3	Sneha	18	74	Pune
8	Arjun	21	72	Lucknow
1	Riya	19	67	Mumbai
6	Vikas	20	65	Hyderabad

6. Create a DataFrame with some missing values intentionally placed in the Marks and City columns. Then:
- Identify missing values using `isnull()`
 - Fill missing marks with the column mean using `fillna()`
 - Drop rows where City is missing using `dropna()`


```
➤ import pandas as pd
import numpy as np
```

```
data = {
    "Name": ["Aman", "Riya", "Karan", "Sneha",
            "Rahul",
            "Priya", "Vikas", "Neha", "Arjun", "Pooja"],
    "Marks": [82, np.nan, 91, 74, np.nan, 79, 65, 95,
              72, 84],
    "City": ["Delhi", "Mumbai", np.nan, "Pune",
            "Chennai",
            np.nan, "Hyderabad", "Bangalore",
            "Lucknow", "Ahmedabad"]
}
```

```
df = pd.DataFrame(data)
```

```
print("Missing Values:")
print(df.isnull())
```

```
df["Marks"].fillna(df["Marks"].mean(),
inplace=True)
```

```
df = df.dropna(subset=["City"])
```

```
print("\nDataFrame After Filling Missing Marks:")
print(df)
```

- output :-

Missing Values:

	Name	Marks	City
0	False	False	False
1	False	True	False
2	False	False	True
3	False	False	False
4	False	True	False
5	False	False	True
6	False	False	False
7	False	False	False
8	False	False	False
9	False	False	False

DataFrame After Filling Missing Marks:

	Name	Marks	City
0	Aman	82.000000	Delhi
1	Riya	80.285714	Mumbai
3	Sneha	74.000000	Pune
4	Rahul	80.285714	Chennai
6	Vikas	65.000000	Hyderabad
7	Neha	95.000000	Bangalore
8	Arjun	72.000000	Lucknow
9	Pooja	84.000000	Ahmedabad

7. read a CSV file (you may create it manually) containing product sales data with columns: Product, Category, Sales, Region.
Perform:

- Group by Category and calculate total Sales using groupby() and agg()
- Find the top 3 products by sales using sort_values() and head()

➤ import pandas as pd

```
data = {  
    "Product": ["Laptop", "Mobile", "Tablet",  
               "Headphones", "Printer",  
               "Monitor", "Keyboard", "Mouse"],  
    "Category": ["Electronics", "Electronics",  
                "Electronics", "Accessories",  
                "Electronics", "Electronics",  
                "Accessories", "Accessories"],  
    "Sales": [50000, 35000, 20000, 10000, 15000,  
             25000, 8000, 6000],  
    "Region": ["North", "South", "East", "West",  
              "North", "South", "East", "West"]  
}
```

```
df = pd.DataFrame(data)
```

```
df.to_csv("sales_data.csv", index=False)
```

```
df = pd.read_csv("sales_data.csv")
```

```
category_sales =  
df.groupby("Category").agg({"Sales": "sum"})
```

```
top_products = df.sort_values(by="Sales",  
ascending=False).head(3)
```

```
print("Total Sales by Category:")  
print(category_sales)
```

```
print("\nTop 3 Products by Sales:")  
print(top_products)
```

- output :-

Total Sales by Category:

Sales

Category

Accessories 24000

Electronics 145000

Top 3 Products by Sales:

	Product	Category	Sales	Region
--	---------	----------	-------	--------

0	Laptop	Electronics	50000	North
---	--------	-------------	-------	-------

1	Mobile	Electronics	35000	South
---	--------	-------------	-------	-------

5	Monitor	Electronics	25000	South
---	---------	-------------	-------	-------

8. Merge two DataFrames:

- DataFrame 1: StudentID, Name, Department
- DataFrame 2: StudentID, Marks, Grade
- Merge on StudentID and display students who scored Grade 'A'

➤ import pandas as pd

```
df1 = pd.DataFrame({
    "StudentID": [101, 102, 103, 104, 105],
    "Name": ["Aman", "Riya", "Karan", "Sneha",
    "Rahul"],
    "Department": ["CSE", "ECE", "IT", "CSE", "ME"]
})
```

```
df2 = pd.DataFrame({
    "StudentID": [101, 102, 103, 104, 105],
    "Marks": [85, 72, 91, 68, 88],
    "Grade": ["A", "B", "A", "C", "A"]
})
```

```
merged_df = pd.merge(df1, df2, on="StudentID")
```

```
grade_a = merged_df[merged_df["Grade"] == "A"]
```

```
print("Merged DataFrame:")
print(merged_df)
```

```
print("\nStudents with Grade 'A':")
print(grade_a)
```

- output :-

Merged DataFrame:

	StudentID	Name	Department	Marks	Grade
0	101	Aman	CSE	85	A

1	102	Riya	ECE	72	B
2	103	Karan	IT	91	A
3	104	Sneha	CSE	68	C
4	105	Rahul	ME	88	A

Students with Grade 'A':

	StudentID	Name	Department	Marks	Grade
0	101	Aman	CSE	85	A
2	103	Karan	IT	91	A
4	105	Rahul	ME	88	A

Section C – Data Visualization

9. Using Matplotlib, create the following plots from a student marks dataset (create the dataset yourself):

- A line plot showing marks trend across 5 subjects
- A bar chart comparing marks of 5 students
- Add proper title, xlabel, ylabel, legend, and grid to both plots

➤ `import matplotlib.pyplot as plt`

```
subjects = ["Math", "Science", "English",
            "Computer", "History"]
marks = [85, 78, 90, 95, 80]
```

```
students = ["Aman", "Riya", "Karan", "Sneha",
            "Rahul"]
student_marks = [82, 75, 91, 88, 79]
```

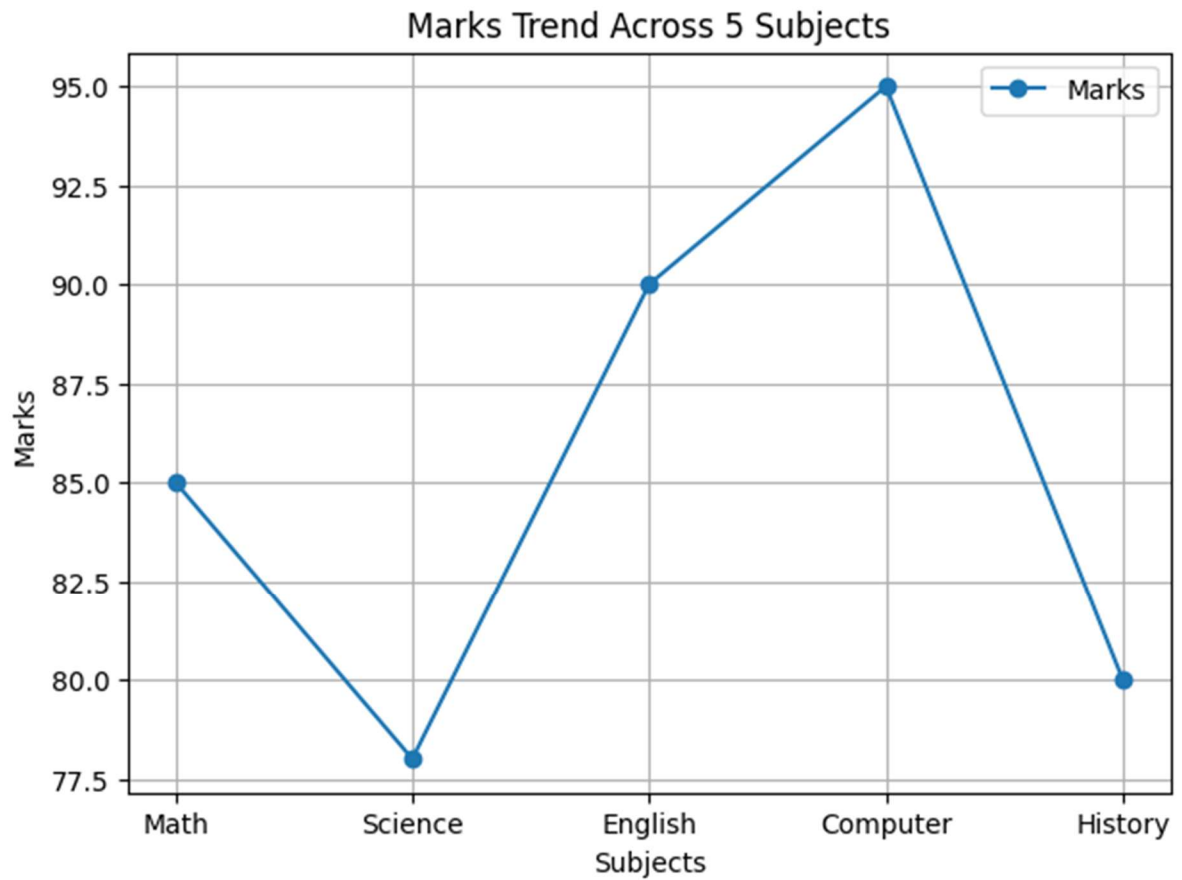
```
plt.figure(figsize=(7, 5))
plt.plot(subjects, marks, marker="o",
label="Marks")
plt.title("Marks Trend Across 5 Subjects")
plt.xlabel("Subjects")
plt.ylabel("Marks")
plt.legend()
plt.grid(True)
plt.show()
```

```
plt.figure(figsize=(7, 5))
plt.bar(students, student_marks, label="Marks")
plt.title("Marks Comparison of 5 Students")
plt.xlabel("Students")
plt.ylabel("Marks")
plt.legend()
plt.grid(True)
plt.show()
```

- output :-

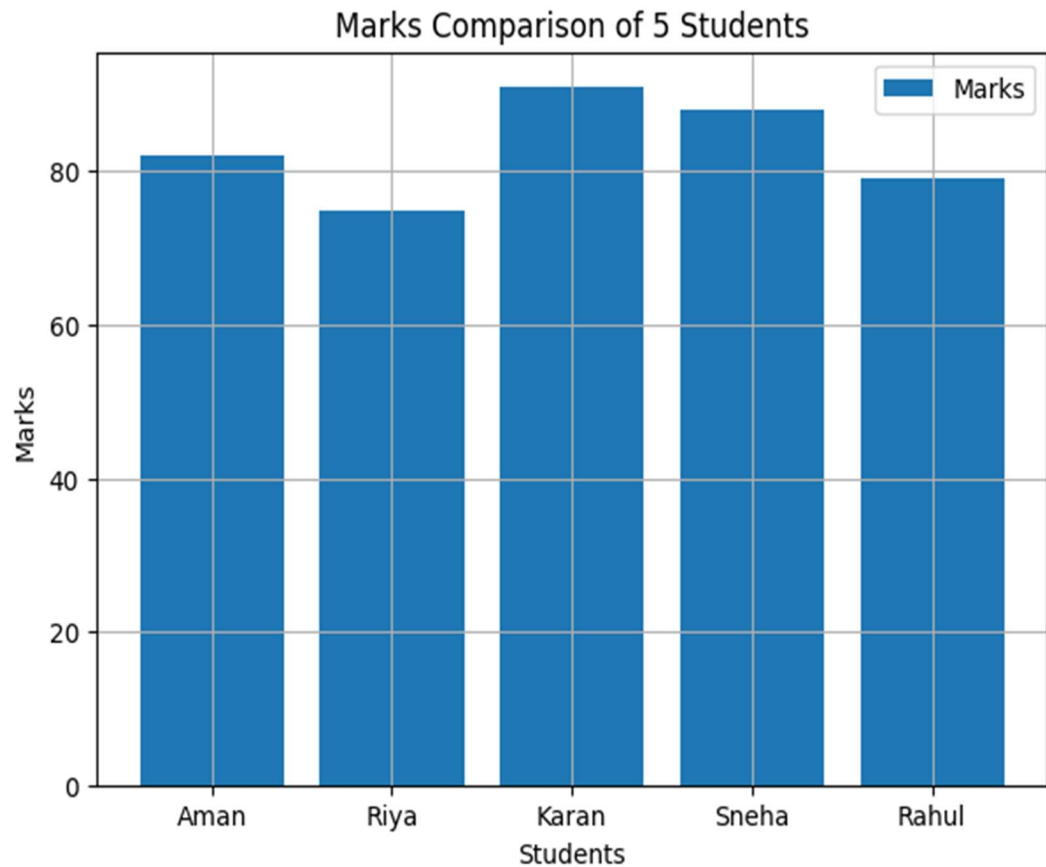
Line Plot Data (Marks Trend Across 5 Subjects):

- Subjects: ['Math', 'Science', 'English', 'Computer', 'History']
- Marks: [85, 78, 90, 95, 80]



- Bar Chart Data (Marks Comparison of 5 Students):
 - Students: ['Aman', 'Riya', 'Karan', 'Sneha', 'Rahul']
 - Marks: [82, 75, 91, 88, 79]

S



10. Using Pandas built-in plotting, load or create a dataset with at least 4 numeric columns. Create:
- A histogram for marks distribution
 - A box plot to visualize spread and outliers
 - A pie chart showing category-wise percentage

➤ `import pandas as pd`
`import matplotlib.pyplot as plt`

```
data = {  
    "Math": [85, 78, 92, 88, 76],  
    "Science": [80, 75, 90, 85, 70],
```

```
"English": [82, 74, 88, 90, 72],  
"Computer": [95, 85, 98, 92, 80]  
}
```

```
df = pd.DataFrame(data)
```

```
df["Math"].plot(kind="hist", title="Marks  
Distribution")  
plt.xlabel("Marks")  
plt.show()
```

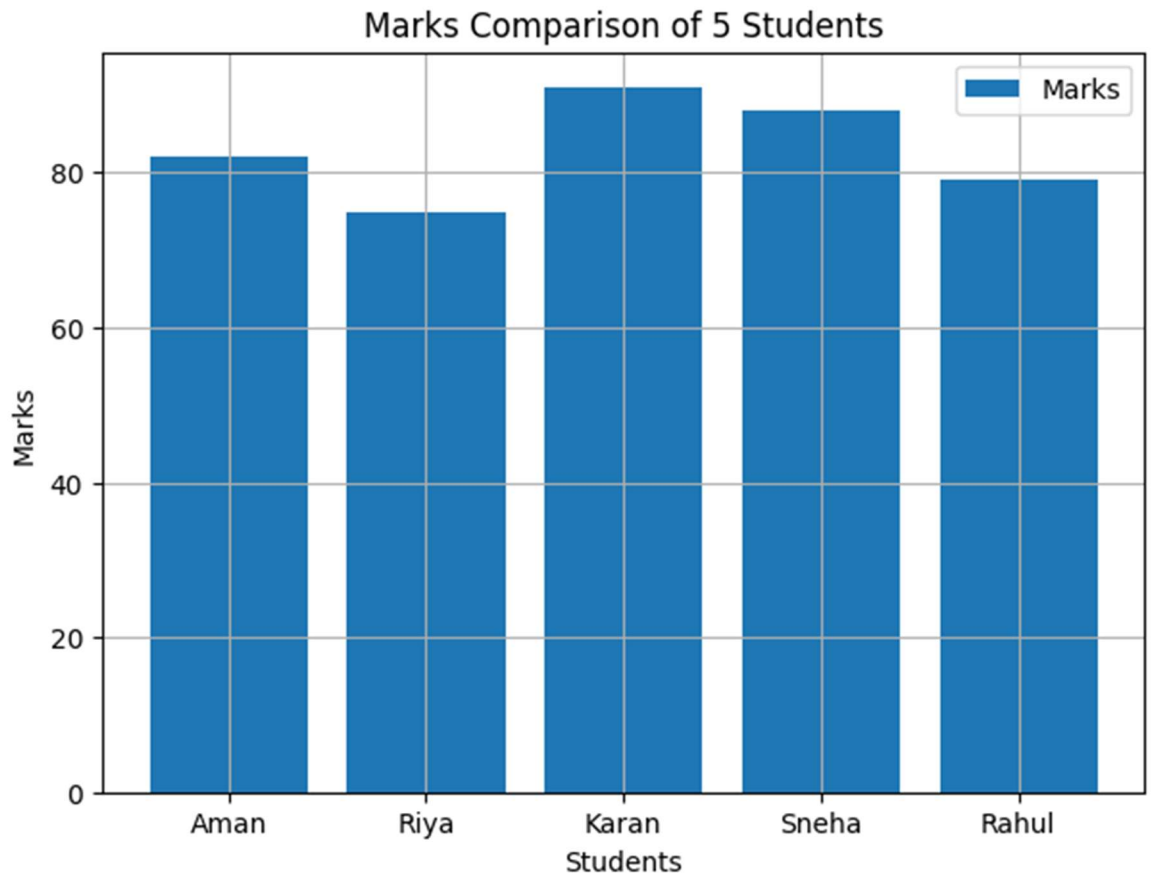
```
df.plot(kind="box", title="Box Plot of Marks")  
plt.ylabel("Marks")  
plt.show()
```

```
category = pd.Series(  
    [40, 30, 20, 10],  
    index=["Math", "Science", "English",  
"Computer"]  
)
```

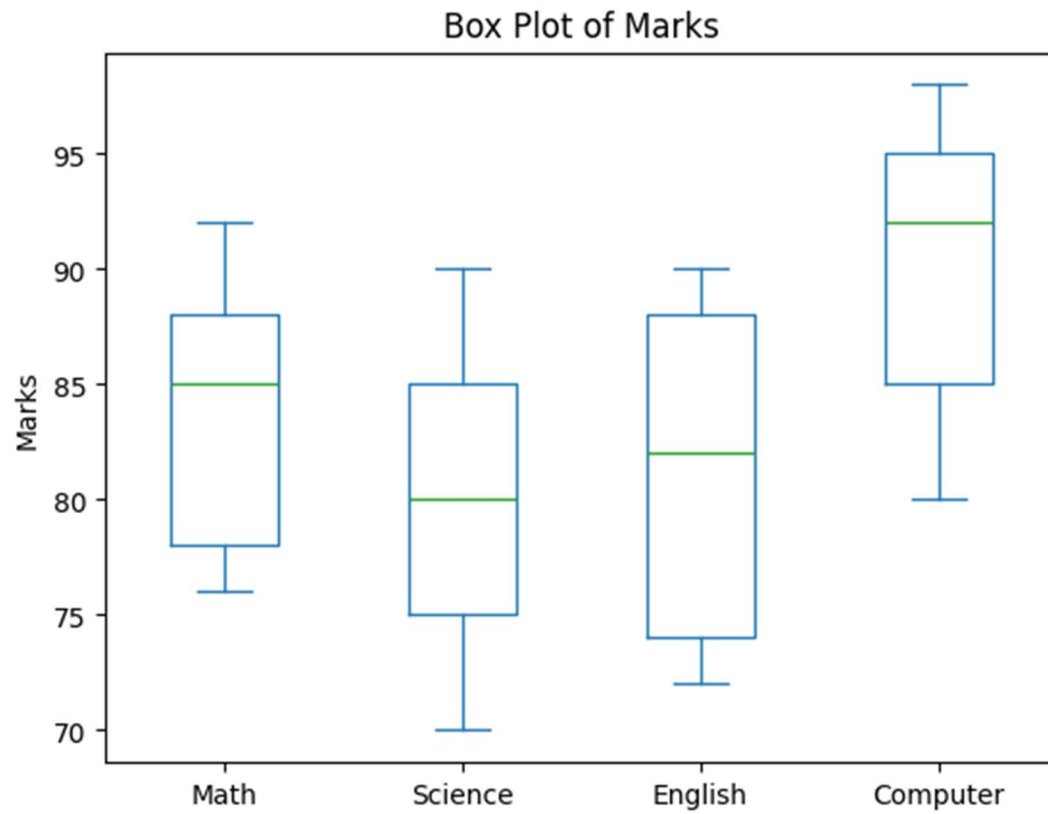
```
category.plot(  
    kind="pie",  
    autopct="%1.1f%%",  
    title="Category-wise Percentage"  
)  
plt.ylabel("")  
plt.show()
```

- output :-

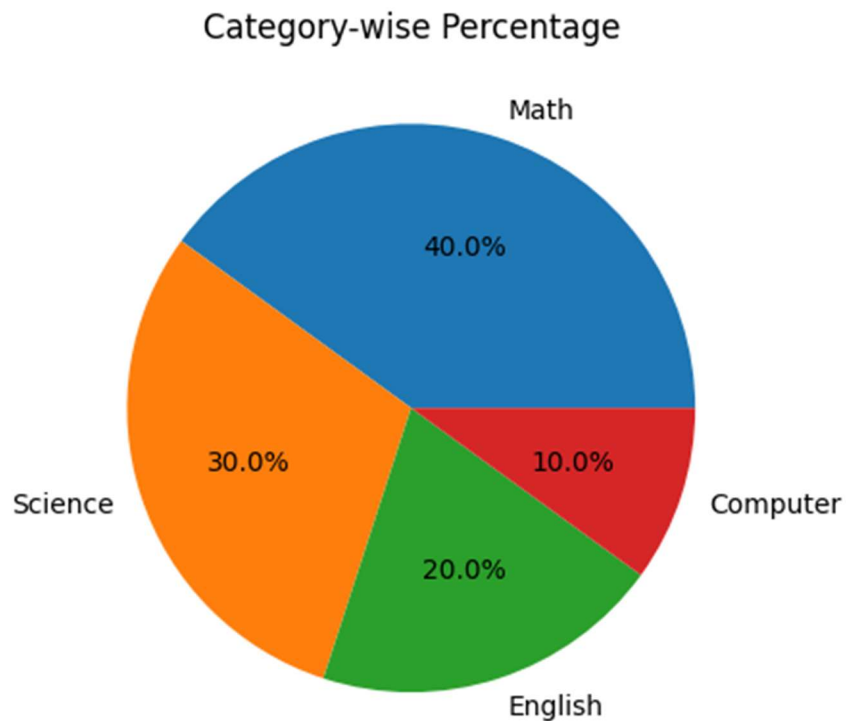
- **Histogram:** Displays the distribution of Math marks.



- **Box Plot:** Shows the spread, median, and any outliers for all four subjects.



- **Pie Chart:** Displays category-wise percentages:
 - Math: 40%
 - Science: 30%
 - English: 20%
 - Computer: 10%



11. Using Seaborn, create a dataset of at least 50 records with columns Age, Salary, Department, Experience. Plot:

- A scatter plot of Age vs Salary with hue as Department
- A heatmap showing correlation between numeric columns
- A bar plot showing average Salary per Department

```
➤ import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

np.random.seed(1)

df = pd.DataFrame({
    "Age": np.random.randint(22, 50, 50),
    "Salary": np.random.randint(25000, 100000, 50),
```

```

        "Department": np.random.choice(["HR", "IT", "Finance",
        "Sales"], 50),
        "Experience": np.random.randint(1, 20, 50)
    })

print(df.head())

sns.scatterplot(data=df, x="Age", y="Salary", hue="Department")
plt.title("Age vs Salary by Department")
plt.show()

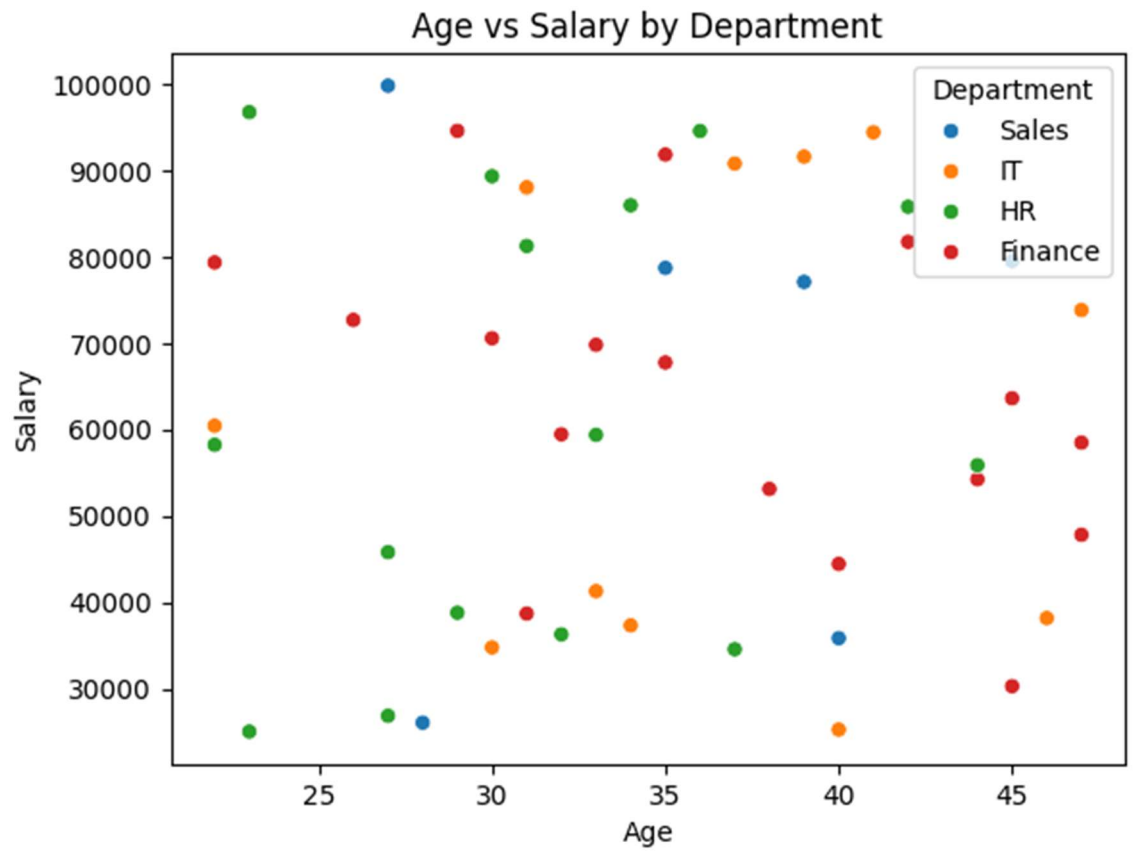
corr = df[["Age", "Salary", "Experience"]].corr()

sns.heatmap(corr, annot=True, cmap="Blues")
plt.title("Correlation Heatmap")
plt.show()

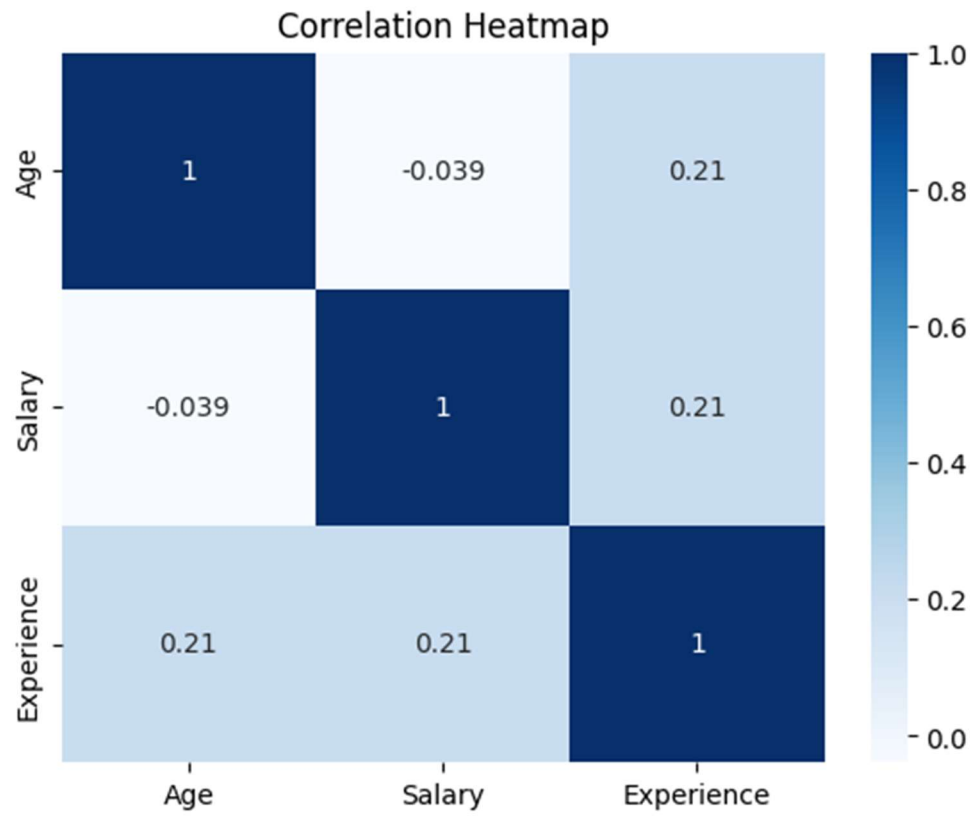
sns.barplot(data=df, x="Department", y="Salary")
plt.title("Average Salary per Department")
plt.show()

```

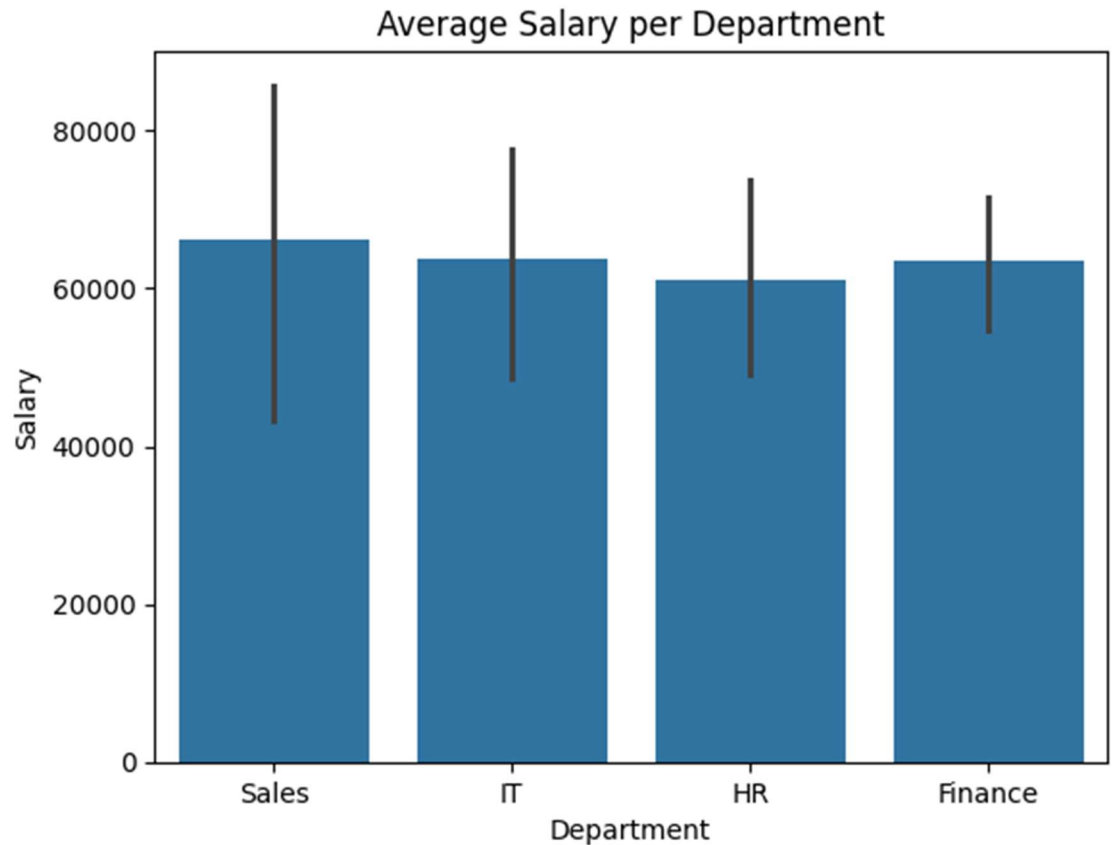
- output :-
 - A scatter plot of Age vs Salary with hue as Department



- A heatmap showing correlation between numeric columns



- A bar plot showing average Salary per Department



12. Create a subplot layout of 2 rows $\times$ 2 columns using Matplotlib containing:

- Line plot
- Scatter plot
- Bar chart
- Histogram All using the same dataset.

Add a main title using `suptitle()`

➤ `import matplotlib.pyplot as plt`

`x = [1, 2, 3, 4, 5]`

`y = [85, 78, 92, 88, 95]`

```
fig, ax = plt.subplots(2, 2, figsize=(10, 8))

ax[0, 0].plot(x, y, marker="o")
ax[0, 0].set_title("Line Plot")

ax[0, 1].scatter(x, y)
ax[0, 1].set_title("Scatter Plot")

ax[1, 0].bar(x, y)
ax[1, 0].set_title("Bar Chart")

ax[1, 1].hist(y)
ax[1, 1].set_title("Histogram")

fig.suptitle("Different Plots Using the Same Dataset", fontsize=16)

plt.tight_layout()
plt.show()
```

- output :-

Different Plots Using the Same Dataset

